



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Tapia Verastegui Venancio Miguel

N° de Cuenta: 320324187

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 04/10/2026

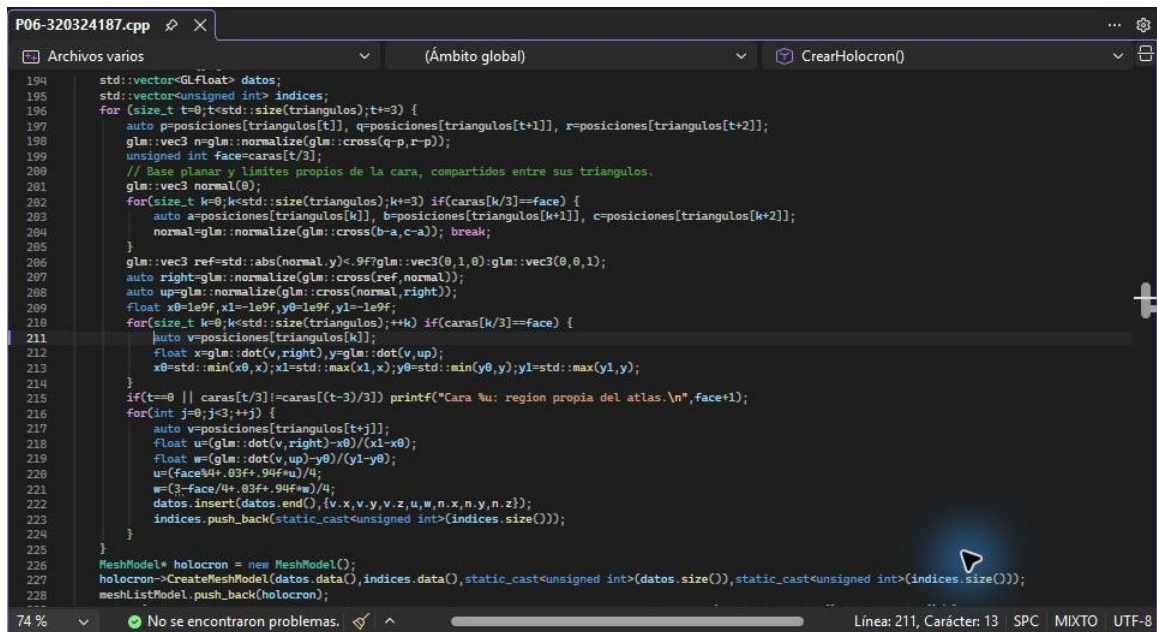
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1. Holocron texturizado por código

Se utilizó la geometría del holocron de los archivos de la práctica. Tiene 14 superficies: 6 principales y 8 triangulares. En C++ se asignó una región del atlas a cada superficie para que todas tengan textura, incluyendo las pequeñas. Las imágenes se acomodaron en GIMP y se guardaron en un PNG.



```
P06-320324187.cpp (Ámbito global) CrearHolocron()
std::vector<GLfloat> datos;
std::vector<unsigned int> indices;
for (size_t t=0; t<std::size(triangulos); t+=3) {
    auto p=posiciones[triangulos[t]], q=posiciones[triangulos[t+1]], r=posiciones[triangulos[t+2]];
    glm::vec3 n=glm::normalize(glm::cross(q-p,r-p));
    unsigned int face=caras[t/3];
    // Base planar y límites propios de la cara, compartidos entre sus triángulos.
    glm::vec3 normal(0);
    for (size_t k=0; k<std::size(triangulos); k+=3) if (caras[k/3]==face) {
        auto a=posiciones[triangulos[k]], b=posiciones[triangulos[k+1]], c=posiciones[triangulos[k+2]];
        normal=glm::normalize(glm::cross(b-a,c-a)); break;
    }
    glm::vec3 ref=std::abs(normal.y)<.9f?glm::vec3(0,1,0):glm::vec3(0,0,1);
    auto right=glm::normalize(glm::cross(ref,normal));
    auto up=glm::normalize(glm::cross(normal,right));
    float x0=1e9f, x1=-1e9f, y0=-1e9f, y1=1e9f;
    for (size_t k=0; k<std::size(triangulos); ++k) if (caras[k/3]==face) {
        auto v=posiciones[triangulos[k]];
        float x=glm::dot(v,right), y=glm::dot(v,up);
        x0=std::min(x0,x), x1=std::max(x1,x), y0=std::min(y0,y), y1=std::max(y1,y);
    }
    if (t==0 || caras[t/3]!=caras[(t-3)/3]) printf("Cara %u: region propia del atlas.\n",face+1);
    for (int j=0; j<3; ++j) {
        auto v=posiciones[triangulos[t+j]];
        float u=(glm::dot(v,right)-x0)/(x1-x0);
        float w=(glm::dot(v,up)-y0)/(y1-y0);
        u=(face%4+.83f+.94f*u)/4;
        w=(3-face/4+.83f+.94f*w)/4;
        datos.insert(datos.end(),{v.x,v.y,v.z,u,w,n.x,n.y,n.z});
        indices.push_back(static_cast<unsigned int>(indices.size()));
    }
}
MeshModel* holocron = new MeshModel();
holocron->CreateMeshModel(datos.data(), indices.data(), static_cast<unsigned int>(datos.size()), static_cast<unsigned int>(indices.size()));
meshListModel.push_back(holocron);
```

Figura 1. Coordenadas UV y creación de la malla en Visual Studio.

La función CrearHolocron recorre los triángulos, calcula las normales y las coordenadas UV. Después guarda posición, UV y normal de cada vértice y crea la malla que se dibuja en OpenGL.

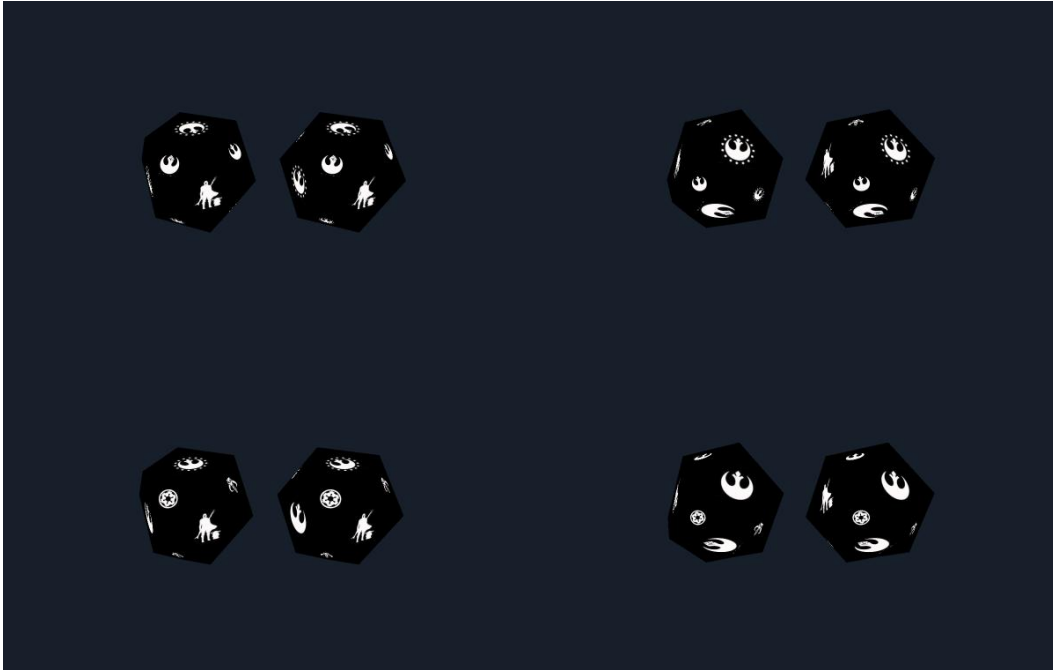
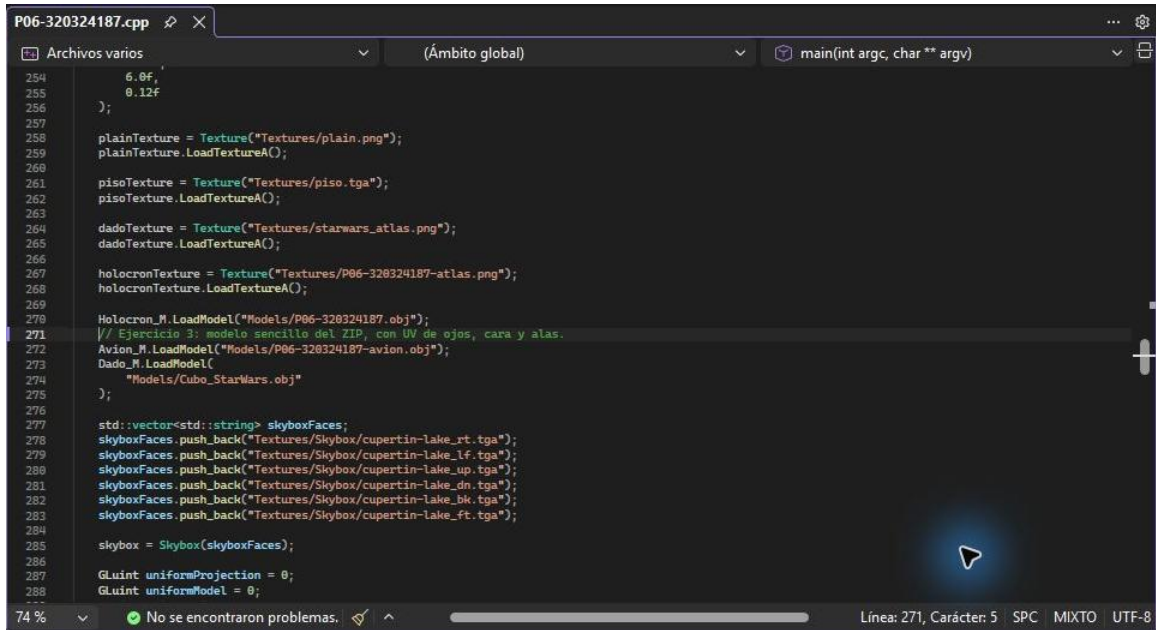


Figura 2. Vistas de comprobación del texturizado de los holocrones.

Ejercicio 2. Holocron importado de Blender

El segundo holocron se preparó en Blender con su mapa UV y el material de imagen. Se exportó como OBJ con su archivo MTL y se cargó en el proyecto de Visual Studio. Así se muestran los dos holocrones dentro de la misma escena: uno construido por código y otro importado.



```
P06-320324187.cpp
Archivos varios (Ámbito global) main(int argc, char ** argv)
254     6.0f,
255     0.12f
256 );
257
258 plainTexture = Texture("Textures/plain.png");
259 plainTexture.LoadTextureA();
260
261 pisoTexture = Texture("Textures/piso.tga");
262 pisoTexture.LoadTextureA();
263
264 dadoTexture = Texture("Textures/starwars_atlas.png");
265 dadoTexture.LoadTextureA();
266
267 holocronTexture = Texture("Textures/P06-320324187-atlas.png");
268 holocronTexture.LoadTextureA();
269
270 Holocron_M.LoadModel("Models/P06-320324187.obj");
271 // Ejercicio 3: modelo sencillo del ZIP, con UV de ojos, cara y alas.
272 Avion_M.LoadModel("Models/P06-320324187-avion.obj");
273 Dado_M.LoadModel(
274     "Models/Cubo_StarWars.obj"
275 );
276
277 std::vector<std::string> skyboxFaces;
278 skyboxFaces.push_back("Textures/Skybox/cupertin-lake_rt.tga");
279 skyboxFaces.push_back("Textures/Skybox/cupertin-lake_lf.tga");
280 skyboxFaces.push_back("Textures/Skybox/cupertin-lake_up.tga");
281 skyboxFaces.push_back("Textures/Skybox/cupertin-lake_dn.tga");
282 skyboxFaces.push_back("Textures/Skybox/cupertin-lake_bk.tga");
283 skyboxFaces.push_back("Textures/Skybox/cupertin-lake_ft.tga");
284
285 skybox = Skybox(skyboxFaces);
286
287 GLuint uniformProjection = 0;
288 GLuint uniformModel = 0;
```

74 % No se encontraron problemas. Línea: 271, Carácter: 5 SPC MIXTO UTF-8

Figura 3. Carga de los modelos OBJ y de las texturas del proyecto.



Figura 4. Vista del holocron preparado en Blender.

El OBJ conserva las coordenadas UV. El material MTL indica qué imagen se utiliza; por eso se entregan juntos el modelo, el material y el atlas. Al dibujarlo se aplican traslación y escala para colocarlo detrás de los cubos.

Ejercicio 3. Avión con textura del personaje

Se importó el avión sencillo del ZIP de E06. La textura se dividió en zonas: ojos en el vidrio frontal, nariz y sonrisa en la parte delantera y los emblemas de estrella de la referencia en las dos alas principales. El resto del avión se dejó azul para mantener el diseño sencillo.

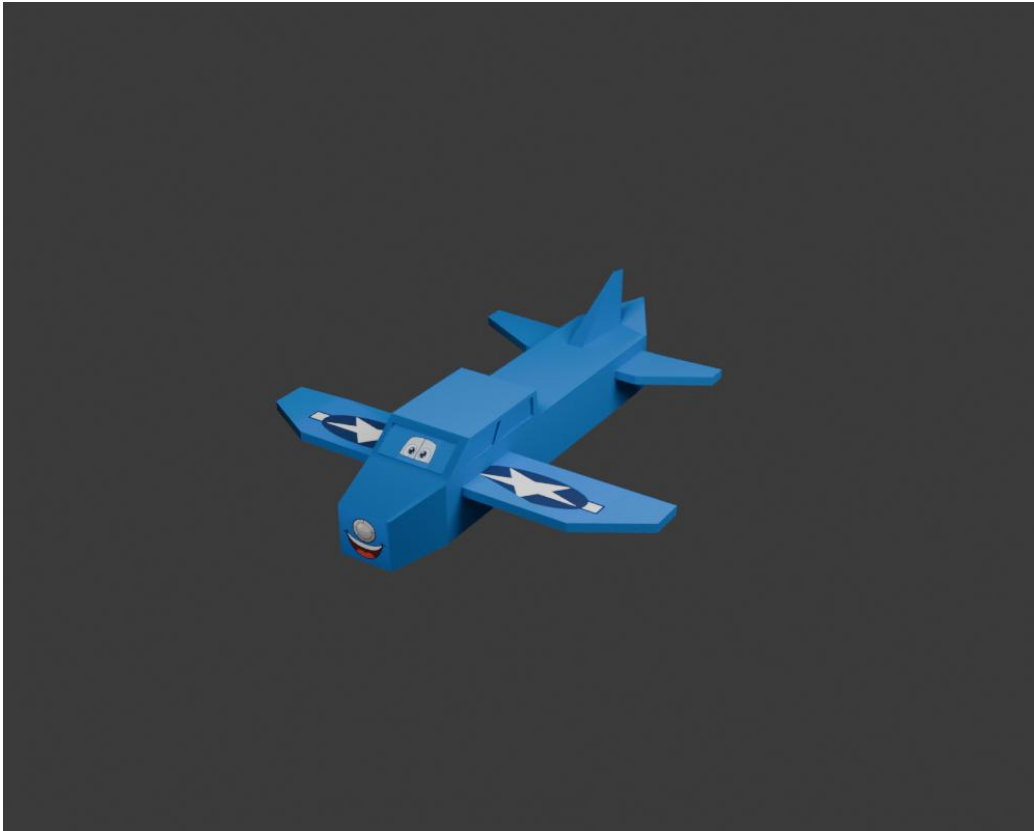


Figura 5. Vista del modelo del avión con sus texturas.

En GIMP se ajustó el tamaño de los ojos y los emblemas para evitar que se vieran estirados. En el mapa UV del modelo se asignó cada zona a la parte que corresponde. El modelo se carga con `Avion_M.LoadModel` y se dibuja con `Avion_M.RenderModel`.

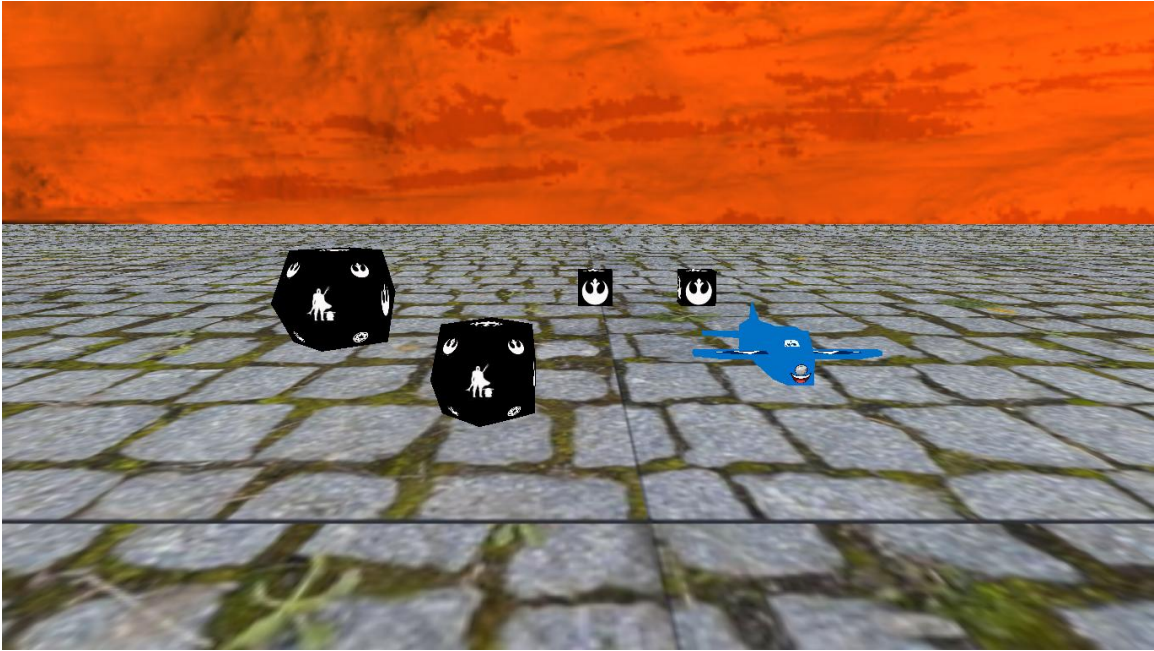


Figura 6. Ejecución en OpenGL: cubos, holocrones y avión en la escena original.

Se conservaron el piso y el skybox. La cámara permite recorrer la escena con WASD y mirar con el mouse. Espacio sube, Ctrl baja y Shift aumenta la velocidad. Tab libera o captura el mouse y Esc cierra el programa.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

1. Algunas superficies del holocron no tenían una región de textura. Se revisó la geometría y se asignaron UV a las 14 superficies, en lugar de considerar solamente las seis principales.

2. Al importar el modelo, la textura no se aplicaba correctamente. Se corrigió la comprobación del índice del material en Model.cpp y se enlazó la textura correspondiente antes de dibujar cada malla.

3. Los ojos y los logos del avión se deformaban por la proporción de las superficies. Se ajustaron sus dimensiones en GIMP y se revisaron las coordenadas UV.

4. Para integrar los modelos se mantuvo como base el proyecto de los dos cubos. Se ajustaron las posiciones de los holocrones y del avión para que se puedan observar sin cambiar el piso ni el skybox.

Después de los ajustes, el proyecto compiló en Release x64 y la ejecución mostró los modelos con sus texturas. No se incluyen capturas de errores porque no se conservó evidencia visual de esas pruebas.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

El ejercicio más complejo fue acomodar las imágenes sobre las caras del holocron y las partes del avión. No basta con cargar el PNG: las coordenadas UV deben indicar qué zona se muestra en cada superficie. La importación del OBJ fue más directa una vez que el material y las rutas estaban correctos.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Como sugerencia, serviría revisar un ejemplo completo del recorrido de una textura desde GIMP y Blender hasta el proyecto de C++, sobre todo las rutas del OBJ, MTL y PNG.

c. Conclusión

En esta práctica se aplicaron texturas por código y mediante un modelo importado. Los dos métodos permiten mostrar imágenes sobre una malla, pero ambos necesitan UV correctas. La escena final reúne los cubos, los dos holocrones y el avión, y se puede recorrer con la cámara.

1. Bibliografía en formato APA

Blender Foundation. (s. f.). UV Editor. Blender Manual.
<https://docs.blender.org/manual/en/latest/editors/uv/index.html>

The GIMP Documentation Team. (s. f.). Scale Layer. GIMP User Manual.
<https://docs.gimp.org/2.10/en/gimp-layer-scale.html>